

Spring 26 DD Onboarding Submission

Name: Saksham Bansal

GT Username: sbansal91

Discord Username: bansalsaksham

1. Design Explanation & Correctness

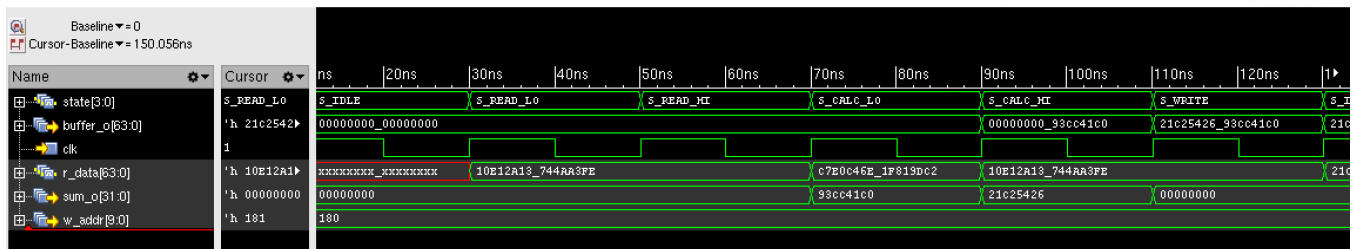
To meet the requirement of performing 64-bit addition using a 32-bit adder and memory data path, I implemented a controller that splits each operation into two 32-bit cycles. The design uses an 8-state Finite State Machine (FSM) to orchestrate the fetch-decode-execute-write sequence.

Logic Flow:

- Lower 32-bits:** The controller first fetches the lower 32 bits of operands A and B (S_READ_LO and S_READ_HI). It computes the sum using the adder and latches the resulting **carry-out** into an internal register. The partial sum is stored in the lower half of the result buffer.
- Upper 32-bits:** The controller then fetches the upper 32 bits of the operands. It performs the addition using the **stored carry-in** from the previous step to ensure mathematical correctness across the 64-bit boundary. This result is stored in the upper half of the result buffer.
- Write Back:** Once the full 64-bit result is assembled in the buffer, the controller performs a single write operation (S_WRITE) to the target memory address.
- Looping:** The controller increments the read/write pointers and a counter. This loop repeats for 128 iterations until all operations are complete.

Performance Verification: The design passed verification with a total execution time of **773 cycles**. This averages to approximately 6 cycles per 64-bit addition (128 additions total), which matches the theoretical minimum for the implemented state sequence.

2. Waveform Analysis



Explanation: The waveform demonstrates a detailed view of a single 64-bit addition cycle:

- Data Fetch (r_data):** During S_READ_LO and S_READ_HI, the **r_data** signal changes values, proving that the controller is successfully fetching the split 32-bit operands from memory.
- Calculation (sum_o):** The adder output **sum_o** updates twice. First, during S_CALC_LO, it computes the lower 32-bit sum (0x93cc41c0). Then, during S_CALC_HI, it computes the upper 32-bit sum (0x21c25426) using the carry-out from the previous step.
- Result Assembly (buffer_o):** The **buffer_o** signal assembles these partial results. You can see the lower half update first, followed by the full 64-bit result (0x21c25426_93cc41c0) becoming valid exactly at the S_WRITE state.
- Write Back:** Finally, the valid 64-bit result is written to the address specified by **w_addr** (0x181), completing the cycle.

3. Simulation Results

```
-----Beginning Simulation!-----  
Time:                0  
-----Initializing Signals-----  
Time:                10000  
-----Finished Simulation!-----  
  
Time:                15490000  
Cycle Count: 773 cycles (from S_IDLE to S_END)  
Simulation complete via $finish(1) at time 15490 NS + 0  
./sym_links/tb_calculator.sv:66      $finish;  
xcelium> exit  
  
coverage setup:  
workdir   : ./cov_work  
dutinst   : tb_calculator(tb_calculator)  
scope     : scope  
testname  : test  
  
coverage files:  
model(design data) : ./cov_work/scope/icc_4b00d6e5_00000000.ucm (reused)  
data          : ./cov_work/scope/test/icc_4b00d6e5_00000000.ucd  
T00L:  xrun(64)      25.03-s003: Exiting on Feb 08, 2026 at 15:58:52 EST  (total: 00:00:00)
```

4. FSM Diagram

